

Redes de Comunicação — Resumo Teórico

Gonçalo Sousa

Atualizado: Semana 1

Conteúdo

Informação da Disciplina	3
Aula 1 — Introdução às Redes de Comunicação	4
O que é a Internet?	4
Extremidade da rede (edge)	5
Núcleo da rede (core)	8
Atrasos, perdas e <i>throughput</i>	12
Camadas protocolares e modelos de serviço	17

Informação da Disciplina

Notas de organização da cadeira (não é matéria de exame, mas é útil ter aqui para consulta rápida em vez de andar a abrir o PDF [AboutRC.pdf](#)).

Docentes: Rui Prior (rcprior@fc.up.pt, gabinete 1.33) e Pedro d'Orey (pdorey@fc.up.pt).

Avaliação: avaliação contínua — 3 testes ao longo do semestre (cada um com componente teórica e prática, cada um vale 30% da nota final) + um trabalho de programação em grupos de 2 (10% da nota final). O exame de recurso substitui os testes.

Requisitos: disciplina introdutória e praticamente auto-contida. Precisa-se de conhecimentos básicos de Java para a parte de programação com sockets (mais para a frente no semestre).

Bibliografia principal: *Computer Networking: A Top-Down Approach*, Jim Kurose e Keith Ross (Addison-Wesley) — os slides das teóricas são traduzidos e adaptados deste livro. Alternativas online gratuitas: *Computer Networking: Principles, Protocols and Practice* (Olivier Bonaventure) e *Fundamentos de Redes de Computadores* (José Legatheaux Martins).

Programa da disciplina (para situar onde estamos): 1. Introdução (arquiteturas de rede, comutação, camadas ISO/OSI e TCP/IP) — **é o que cobre esta Aula 1** — 2. Camada de Aplicação (HTTP, FTP, SMTP, DNS, sockets) — 3. Camada de Transporte (UDP, TCP, controlo de fluxo/ congestionamento) — 4. Camada de Rede (encaminhamento, IP, ICMP, DHCP, NAT, RIP/OSPF/BGP) — 5. Camada de Ligação de Dados (detecção/correção de erros, Ethernet, ARP, redes sem fios).

Aula 1 — Introdução às Redes de Comunicação

O que é a Internet?

Visão estrutural: hosts, ligações e routers

A Internet pode ser descrita de duas formas complementares: pelos seus **componentes** (visão estrutural, “debaixo do capô”) ou pelo **serviço** que presta às aplicações.

Do ponto de vista estrutural, a Internet é composta por:

- **Terminais** (*hosts* ou *end systems*): milhões de dispositivos ligados à rede — não só PCs e servidores, mas cada vez mais objetos do dia-a-dia (“IoT” — frigoríficos, torradeiras, carros, câmaras de vigilância, telefones IP). Todos correm **aplicações em rede**.
- **Ligações** (*links*): o meio por onde os dados viajam — fibra, cobre, rádio, satélite. Cada ligação tem uma **taxa de transmissão** (também chamada capacidade ou, informalmente, “largura de banda”), medida em bits por segundo.
- **Routers**: dispositivos que reenviam pacotes de dados, encaminhando-os em direção ao seu destino.

Definição — Internet

A Internet é uma “**rede de redes**”: uma coleção de redes independentes interligadas, vagamente organizada de forma hierárquica (ver secção sobre a estrutura da Internet, mais à frente). Distingue-se a **Internet pública** (a que todos conhecemos) de **intranets privadas**, que usam a mesma tecnologia mas não estão acessíveis a partir da Internet pública (ou só o estão de forma restrita).

Toda a comunicação na Internet é regida por **protocolos** — por exemplo, TCP e IP controlam o envio e receção de pacotes; HTTP rege a Web; Ethernet rege a transferência de dados numa rede local. Estes protocolos são definidos em documentos chamados **RFC** (*Request for Comments*), geridos pelo **IETF** (*Internet Engineering Task Force*).

Visão de serviços

Vista do lado das aplicações, a Internet é uma **infraestrutura de comunicação** que possibilita a existência de aplicações distribuídas: Web, email, VoIP, jogos em rede, partilha de ficheiros, etc. Estas aplicações não precisam de saber como os bits chegam ao destino — só precisam de um serviço de comunicação. A Internet oferece dois tipos:

- **Entrega de dados fiável**, extremo-a-extremo (usada por TCP);
- **Entrega de dados não-fiável**, do tipo “melhor esforço” (*best effort*, usada por UDP).

Estes dois modelos de serviço são explicados em detalhe na secção seguinte.

O que é um protocolo?

Definição — Protocolo

Um **protocolo** define:

- o **formato** das mensagens trocadas;
- a **ordem** de envio e receção dessas mensagens entre as entidades da rede;
- as **ações** a efetuar na transmissão ou na receção de uma mensagem (ou em resposta a outro evento, como um temporizador que expira).

É exatamente como um protocolo humano: quando dizemos “Olá” a alguém e recebemos “Olá” de volta antes de perguntar as horas, estamos a seguir um protocolo (um formato de mensagem — uma saudação — e uma ordem — a saudação antes do pedido).

Exemplo resolvido

Paralelo entre um protocolo humano e um protocolo de rede.

Do lado humano: a pessoa A diz “Olá” à pessoa B; B responde “Olá”; só depois é que A pergunta “Tens horas?” e B responde “São 2:00”. Se B, em vez de “Olá”, respondesse logo com a hora, a comunicação seria estranha — quebrava-se o protocolo esperado (a ordem das mensagens importa).

Do lado da rede, o mesmo tipo de troca acontece quando um browser vai buscar uma página web:

1. O cliente (o browser) envia um **pedido de ligação TCP** ao servidor.
2. O servidor responde com uma **resposta de ligação TCP**, confirmando que aceita estabelecer a conexão (isto é o “handshake” — o estabelecimento prévio da ligação, discutido na secção seguinte).
3. Só depois de a ligação estar estabelecida é que o cliente envia o pedido real, por exemplo **GET** `http://www.exemplo.com/pagina.html`.
4. O servidor responde enviando o <ficheiro> pedido.

Tal como no protocolo humano, a **ordem importa**: o cliente não pode enviar o **GET** antes de a ligação TCP estar estabelecida, tal como B não responde “são 2 horas” antes de ouvir a pergunta. E o **formato** também importa: **GET** `http://...` é uma mensagem com uma sintaxe específica que o servidor sabe interpretar — se o cliente enviasse uma mensagem com outro formato, o servidor não saberia o que fazer com ela.

Extremidade da rede (edge)

Terminais e modelos de aplicação

Os terminais (*hosts*) correm as aplicações — Web, email, etc. — e situam-se na periferia (“extremidade”) da rede. Há dois modelos fundamentais de organização das aplicações:

Definição

Modelo cliente/servidor: existe um terminal **cliente** que envia pedidos, e um terminal **servidor**, sempre ligado, que responde a esses pedidos e fornece o serviço. Exemplos: um browser a pedir uma página a um servidor Web; um cliente de email a comunicar com um servidor de email.

Modelo *peer-to-peer* (p2p): uso mínimo (ou nenhum) de servidores dedicados — os próprios terminais comunicam diretamente uns com os outros e trocam entre si o papel de “cliente” e “servidor” consoante a necessidade. Exemplos: Skype, BitTorrent.

Atenção

Nota adicional (não estava explícito nos slides): a distinção cliente/servidor vs. p2p não é sobre *quem inicia* a ligação — em ambos os casos alguém tem de iniciar. A diferença está na **arquitetura**: no modelo cliente/servidor há uma entidade fixa e sempre disponível (o servidor) da qual todos os clientes dependem; no p2p não há esse ponto central — cada par pode agir ora como cliente, ora como servidor, consoante o pedido, e o sistema continua a funcionar mesmo que alguns pares saiam da rede (é por isso que o BitTorrent consegue distribuir ficheiros grandes sem precisar de um servidor central potente).

Serviços de transferência de dados: TCP vs UDP

Os terminais também disponibilizam o **serviço de transporte** que as aplicações usam para comunicar entre si. Há dois serviços na Internet, com propriedades muito diferentes:

Definição — Serviço TCP (Transmission Control Protocol, RFC 793)

Objetivo: transferência de dados entre terminais, com garantias fortes.

- **Handshaking:** antes de trocar dados, os dois terminais estabelecem a ligação previamente — cria-se “estado” (contexto) em ambos os lados que vai ser usado durante toda a comunicação.
- Transferência de dados **fiável** (nenhum dado se perde — se um pacote se perde, é detetado e retransmitido), **ordenada** (os dados chegam pela mesma ordem em que foram enviados) e organizada como um **fluxo contínuo de bytes** (*byte-stream* — a aplicação não vê “pacotes”, vê um fluxo).
- **Controlo de fluxo:** o emissor nunca envia mais dados do que o recetor consegue processar (evita que o recetor fique sobrecarregado).
- **Controlo de congestionamento:** os emissores adaptam a sua taxa de envio à capacidade disponível na rede (evita sobrecarregar a própria rede).

Aplicações típicas que usam TCP: HTTP (Web), FTP (transferência de ficheiros), Telnet (login remoto), SMTP (email) — todas precisam de que os dados cheguem completos e pela ordem certa.

Definição — Serviço UDP (User Datagram Protocol, RFC 768)

Objetivo: o mesmo — transferência de dados entre terminais — mas **sem** nenhuma das garantias do TCP:

- **Sem conexão:** não há handshaking prévio, envia-se logo.
- Transferência de dados **não-fiável** (pacotes podem perder-se e nunca mais são retransmitidos), **sem controlo de fluxo** e **sem controlo de congestionamento**.

Aplicações típicas: videoconferência, *streaming*, DNS, telefone via Internet — aplicações em que chegar tarde é pior do que não chegar (não vale a pena esperar para retransmitir um pacote de voz atrasado), ou em que a perda ocasional de um pacote não é grave.

Atenção

Nota adicional (não estava explícito nos slides): é fácil pensar “TCP é sempre melhor porque tem mais garantias”, mas essas garantias têm um custo: handshaking, retransmissões e controlo de fluxo/congestionamento introduzem **atraso extra**. Para uma chamada de voz em tempo real, um pacote perdido ou 200ms de atraso extra à espera de uma retransmissão é pior do que simplesmente ignorar o pacote perdido e seguir em frente — daí UDP ser preferido nesses casos. A escolha entre TCP e UDP é sempre um compromisso entre **fiabilidade** e **atraso/simplicidade**, não uma questão de qual é “o melhor” em absoluto. Isto será revisitado em detalhe na camada de transporte.

Redes de acesso residenciais

A **rede de acesso** é o que liga um terminal ao primeiro *router* da rede (o *edge router*). Os aspetos relevantes de qualquer rede de acesso são a sua **capacidade** (partilhada ou dedicada) e a sua **latência**.

Definição

Acesso por cabo (HFC — *hybrid fiber/coaxial*): a mesma infraestrutura de cabo coaxial da TV por cabo transporta também dados. Usa-se **multiplexagem em frequência** (FDM): canais de TV e canais de dados (subida e descida) são transmitidos em bandas de frequência diferentes no mesmo cabo — daí ser possível ver TV e ter Internet ao mesmo tempo pelo mesmo fio. É uma ligação **assimétrica** (mais capacidade a descarregar do que a enviar, tipicamente até ~30Mbps a descer e ~2Mbps a subir) e o cabo é **partilhado** entre todas as casas ligadas ao mesmo segmento — ou seja, o débito real de cada casa depende de quantos vizinhos estão a usar a rede ao mesmo tempo.

Fibra até casa (FTTH — *fiber to the home*): ligação ótica direta (ou quase) da central até casa, através de um *splitter* ótico que distribui o sinal de uma fibra para várias casas (rede **PON**, passiva) ou com eletrónica ativa por casa (**AON**). Capacidade muito superior à do cabo e a distâncias maiores, e permite integrar telefone e televisão na mesma fibra.

Atenção

Nota adicional (não estava explícito nos slides): a diferença central entre cabo (HFC) e fibra (FTTH) não é só “a fibra é mais rápida” — é que no **cabo o meio físico é partilhado** entre vizinhos (o mesmo segmento de cabo coaxial serve várias casas, multiplexado em frequência), enquanto na **fibra (PON)** cada casa tem uma fibra dedicada até ao *splitter*, e a partilha (se existir) acontece de forma mais controlada mais a montante, na central. Isto tem implicações práticas: numa rede de cabo, se muitos vizinhos estiverem a fazer *streaming* ao mesmo tempo, o teu débito pode descer — na fibra, isso é bem menos provável.

Redes de acesso empresariais e sem fios

Definição

LANs empresariais (Ethernet): nas empresas e universidades, os terminais ligam-se ao *router* através de uma rede local Ethernet, a diferentes velocidades padronizadas (10 Mbps, 100 Mbps, 1 Gbps, 10 Gbps). Na configuração moderna, os terminais ligam-se a um **switch Ethernet** (em vez de partilharem um cabo comum, como acontecia nas Ethernet mais antigas).

Redes sem fios (*wireless*): uma rede de acesso sem fios partilhada liga os terminais a um *router* através de um **ponto de acesso**. Nas LANs sem fios (WiFi, normas 802.11b/g/n/ac/ax/be), o débito máximo teórico varia entre 11 Mbps (802.11b, a norma mais antiga) e 46 Gbps (802.11be, a mais recente). Nas redes de longa distância sem fios (WAN sem fios), operadas por operadoras de telecomunicações, os débitos vão até ~300 Mbps em 4G (LTE Avançado) e até 20 Gbps em 5G.

Definição

Meio físico — guiado vs. não guiado. Um bit propaga-se entre um par emissor/recetor através de uma **ligação física**. Há dois tipos:

- **Meio guiado:** o sinal propaga-se por um meio sólido — par de cobre entrançado (categorias Cat.5/5e/6/6a, suportando de 100 Mbps a 10 Gbps de Ethernet consoante a categoria), cabo coaxial, ou fibra ótica.
- **Meio não guiado:** o sinal propaga-se livremente pelo ar/espaco — ondas de rádio. Tipos de ligação rádio: microondas terrestres (até 620 Mbps), LAN sem fios (WiFi, até 46 Gbps), redes de longa distância (redes móveis, até 20 Gbps em 5G) e satélite (geossíncrono ou de baixa altitude — os satélites de baixa altitude, como a constelação Starlink, têm latência muito menor: cerca de 25ms contra 270ms dos geossíncronos, mas capacidade menor por satélite, até ~220 Mbps).

O sinal transportado num meio pode ser transmitido em **banda-base** (um único canal ocupa toda a capacidade do meio, sem modulação — é o caso da Ethernet original em cabo coaxial) ou **modulado** (o sinal digital modula uma onda portadora em frequência, amplitude ou fase — permitindo usar múltiplos canais no mesmo meio, como no HFC).

Núcleo da rede (core)

O **núcleo da rede** é a malha de *routers* interligados que liga as diferentes redes de acesso entre si. A questão fundamental do núcleo da rede é: **como se efetua a transferência de dados através da rede?** Há duas respostas possíveis — comutação de circuitos e comutação de pacotes.

Comutação de circuitos

Definição — Comutação de circuitos

Na comutação de circuitos (como na rede telefônica tradicional), os recursos necessários ao longo de todo o percurso (capacidade nas ligações, capacidade de comutação nos nós) são **reservados extremo-a-extremo antes** de a comunicação começar — é necessário **estabelecer previamente** a “chamada”. Uma vez reservados, esses recursos ficam garantidos e **não são partilhados** com mais ninguém durante a duração da chamada, mesmo que não haja nada para transmitir nesse momento (desperdício de recursos). Em compensação, o **desempenho fica garantido** — não há congestionamento possível dentro daquele circuito.

Os recursos são divididos em “parcelas” atribuídas às chamadas, através de duas técnicas de multiplexagem:

- **FDM** (*Frequency Division Multiplexing*): a capacidade total é dividida em bandas de frequência, uma por utilizador — cada utilizador usa a “sua” banda em permanência, durante toda a chamada.
- **TDM** (*Time Division Multiplexing*): o tempo é dividido em ciclos, e cada ciclo é dividido em *slots* — cada utilizador usa o “seu” *slot* em cada ciclo, sempre o mesmo, de forma periódica e determinística.

Exemplo resolvido

Cálculo do tempo de transferência num circuito. Quanto tempo demora a enviar um ficheiro de 640 000 bits do terminal A para o terminal B, numa rede de comutação de circuitos, sabendo que:

- as ligações ao longo do percurso têm capacidade de 2,048 Mbps;
- cada ligação usa TDM com 32 *slots* por ciclo (ou seja, a capacidade total da ligação é dividida em 32 fatias iguais, uma por utilizador possível);
- são necessários 500 ms para estabelecer o circuito extremo-a-extremo (o *handshaking* prévio, antes de poder começar a enviar dados).

Passo 1 — capacidade efetivamente atribuída ao circuito. Como a capacidade da ligação (2,048 Mbps) é dividida em 32 *slots* iguais, e o nosso circuito só usa 1 desses *slots*, a capacidade que o nosso circuito efetivamente tem disponível é:

$$\frac{2,048 \text{ Mb/s}}{32} = 64 \text{ kb/s}$$

Passo 2 — tempo de transmissão dos dados, usando essa capacidade de 64 kb/s:

$$\frac{640\,000 \text{ bits}}{64\,000 \text{ bits/s}} = 10 \text{ s}$$

Passo 3 — somar o tempo de estabelecimento do circuito (os 500 ms gastos antes de o circuito estar pronto, que têm de ser somados ao tempo de transmissão dos dados, porque acontecem em sequência, não em simultâneo):

$$10 \text{ s} + 500 \text{ ms} = \mathbf{10,5 \text{ s}}$$

(Na prática, seria ainda preciso somar o tempo de propagação do sinal ao longo do circuito — ver a secção “Atrasos, perdas e *throughput*” mais à frente, onde este conceito é aprofundado.)

Comutação de pacotes

Definição — Comutação de pacotes

Na comutação de pacotes, o fluxo de dados extremo-a-extremo é dividido em pequenos pedaços chamados **pacotes** (dados da aplicação + informação de controlo/cabeçalho). Ao contrário da comutação de circuitos:

- **Não há reserva prévia de recursos.** Pacotes de diferentes utilizadores partilham os recursos da rede à medida da necessidade — é a chamada **multiplexagem estatística**: a sequência de pacotes de cada utilizador não segue nenhum padrão fixo (ao contrário do TDM, em que cada emissor usa sempre o mesmo *slot*, de forma determinística e periódica); a capacidade é distribuída consoante a procura em cada instante.
- Cada transmissão de um pacote usa a **capacidade total** da ligação (não uma fração fixa, como no TDM).
- Como não há reserva, pode haver **contenda** no acesso: se a procura de recursos por parte de vários utilizadores exceder a oferta disponível num dado instante, os pacotes ficam em **fila de espera** e pode ocorrer **congestionamento**.

Definição — Store-and-forward

Os pacotes são transferidos **salto a salto** (de *router* em *router*) ao longo do percurso. Em cada nó (*router*), aplica-se a regra *store-and-forward*: o *router* tem de **receber o pacote completo** antes de poder começar a reenviá-lo para o salto seguinte — não pode começar a retransmitir um pacote que ainda não recebeu por inteiro. Isto introduz um **atraso de transmissão** em cada salto: um pacote de L bits demora L/R segundos a ser “colocado” numa ligação de capacidade R bits/s.

Como cada pacote é encaminhado de forma **independente**: cada nó escolhe o próximo salto para cada pacote individualmente, pelo que pacotes com o mesmo destino não têm de seguir a mesma rota (embora normalmente sigam), podem chegar fora de ordem, e podem mesmo perder-se ou ser danificados em trânsito — cabe às máquinas de origem e destino detetar e corrigir estas situações (é isso que o TCP faz, por exemplo).

Exemplo resolvido

Atraso *store-and-forward* ao longo de vários saltos. Um pacote de $L = 10$ Mbits é enviado de um terminal, através de dois *routers* intermédios (logo, 3 ligações/saltos no total), até ao terminal de destino. Todas as ligações têm capacidade $R = 2$ Mb/s. Qual o atraso total até o pacote chegar completo ao destino (ignorando atraso de propagação)?

Passo 1 — atraso de transmissão por salto. Em cada ligação, o pacote demora $L/R = 10\,000\,000/2\,000\,000 = 5$ segundos a ser totalmente colocado na ligação.

Passo 2 — porque é que os atrasos se somam (não se sobrepõem). Por causa da regra *store-and-forward*, o primeiro *router* só pode começar a reenviar o pacote para o segundo *router* depois de o ter recebido **na íntegra** — ou seja, depois dos 5 segundos que o terminal de origem demorou a transmiti-lo todo. Só nesse momento é que o primeiro *router* começa a transmiti-lo na segunda ligação, o que demora outros 5 segundos. E só depois disso é que o segundo *router* pode começar a enviá-lo para o destino, mais 5 segundos. Os três intervalos de 5 segundos acontecem **um a seguir ao outro**, nunca em simultâneo (cada um só pode começar quando o anterior termina).

Passo 3 — atraso total:

$$3 \times \frac{L}{R} = 3 \times 5\text{s} = 15\text{ s}$$

De forma geral, para N saltos: atraso total = $N \cdot L/R$ (assumindo atraso de propagação nulo — na prática seria preciso somar também o atraso de propagação de cada ligação, tratado na próxima secção).

Comutação de pacotes vs. comutação de circuitos

A comutação de pacotes é a que permite **mais utilizadores** partilhar a mesma rede, precisamente porque não reserva recursos que depois ficam desperdiçados quando um utilizador está inativo.

Exemplo resolvido

Exemplo comparativo. Considere uma ligação de 1 Mb/s. Cada utilizador está “ativo” (a transmitir a 100 kb/s) apenas 10% do tempo, e inativo (sem transmitir nada) nos restantes 90%.

Com comutação de circuitos: cada utilizador precisa de um circuito dedicado de 100 kb/s reservado o tempo todo (mesmo quando inativo). Com uma ligação de 1 Mb/s, cabem exatamente $1000/100 = 10$ utilizadores — nem mais um, porque não há partilha possível.

Com comutação de pacotes: como os recursos só são usados quando há dados para enviar, é possível admitir **muito mais** de 10 utilizadores, desde que seja improvável que mais de 10 estejam ativos **ao mesmo tempo**. Com 35 utilizadores, a probabilidade de haver mais de 10 ativos em simultâneo é de apenas aproximadamente 0,0004 (ou seja, praticamente nunca acontece) — logo, a rede consegue servir bem 35 utilizadores com a mesma capacidade que só serviria 10 em comutação de circuitos.

Atenção

Nota adicional (não estava explícito nos slides): de onde vem o valor 0,0004? Cada utilizador está ativo com probabilidade $p = 0,1$, independentemente dos outros. O número de utilizadores ativos em simultâneo, entre $N = 35$, segue uma **distribuição binomial** $X \sim \text{Bin}(N, p)$. Queremos a probabilidade de mais de 10 estarem ativos ao mesmo tempo:

$$P(X > 10) = 1 - \sum_{k=0}^{10} \binom{35}{k} p^k (1-p)^{35-k}$$

Cada termo do somatório dá a probabilidade de exatamente k utilizadores estarem ativos: $\binom{35}{k}$ conta de quantas formas se pode escolher quais k , dos 35, estão ativos; p^k é a probabilidade de esses k estarem mesmo ativos; $(1-p)^{35-k}$ é a probabilidade de todos os restantes estarem inativos. Somando essa probabilidade para $k = 0$ até $k = 10$ obtém-se a probabilidade de **no máximo** 10 estarem ativos; 1 menos esse valor dá a probabilidade de **mais de** 10 estarem ativos. Na prática, este cálculo faz-se com uma calculadora ou software (não se espera que se some os 11 termos à mão num teste) — o que importa perceber é o modelo (binomial) e a ideia de que, com muitos utilizadores independentes, é estatisticamente muito improvável que “todos” estejam ativos ao mesmo tempo.

Frequente em exame

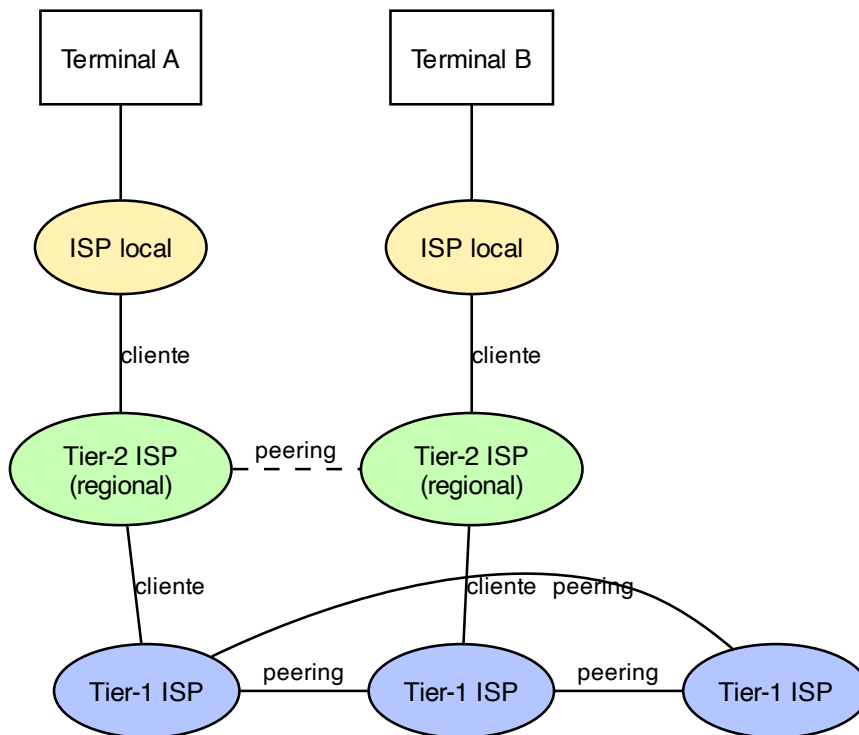
A comutação de pacotes não é uma “panaceia” — é ótima para tráfego em rajadas (*bursty*, como a navegação Web) porque permite partilha de recursos sem estabelecimento prévio de chamada, mas pode sofrer **congestionamento**: atrasos e perdas de pacotes quando a procura excede a capacidade disponível num dado momento. Por isso são necessários **protocolos** (como o TCP) para garantir transferência de dados fiável e para controlar o congestionamento. Garantir à comutação de pacotes um desempenho tão previsível quanto o de um circuito dedicado (para fluxos áudio/vídeo, por exemplo) continua a ser, em geral, um problema em aberto. Isto é um ponto frequentemente sublinhado como importante — a ideia de que comutação de pacotes tem *trade-offs*, não é estritamente “melhor” que comutação de circuitos.

Estrutura da Internet: hierarquia de ISPs

A Internet é organizada de forma **aproximadamente hierárquica**, uma “rede de redes” de fornecedores de acesso (ISPs — *Internet Service Providers*):

- **ISPs Tier-1:** cobertura internacional, interligados entre si “todos para todos” (uma espécie de clique), através de ligações privadas chamadas **peering** — não pagam uns aos outros, trocam tráfego mutuamente porque ambos beneficiam.

- **ISPs Tier-2:** ISPs de menor dimensão, nacionais ou regionais. Ligam-se a um ou mais ISPs Tier-1 (e possivelmente a outros ISPs Tier-2). Um ISP Tier-2 tipicamente **paga** a um Tier-1 para ter conectividade ao resto da Internet — o Tier-2 é **cliente** do Tier-1 nessa relação. Alguns pares de ISPs Tier-2 também estabelecem ligações de *peering* diretamente entre si.
- **ISPs Tier-3 e locais:** as redes de acesso mais próximas dos utilizadores finais — são clientes dos ISPs de nível mais elevado, ligando-se através deles ao resto da Internet.



Um pacote enviado de um Terminal A para um Terminal B atravessa tipicamente várias destas redes: sai do ISP local de A, passa pelo ISP Tier-2 de A, eventualmente por um ou mais ISPs Tier-1 (se A e B não partilharem o mesmo Tier-2), desce pelo ISP Tier-2 de B e chega ao ISP local de B.

Atenção

Nota adicional (não estava explícito nos slides): repara na diferença entre uma ligação **cliente** e uma ligação de **peering** no diagrama: numa relação cliente, o ISP “de baixo” paga ao ISP “de cima” para ter conectividade ao resto da Internet (é uma relação assimétrica, como pagar a um fornecedor); numa relação de *peering*, dois ISPs do mesmo nível trocam tráfego diretamente entre si porque ambos beneficiam (evita ter de passar pelo Tier-1, que seria mais lento/caro) — é uma relação simétrica, sem pagamento entre as partes. Isto explica, por exemplo, porque é que às vezes um pacote entre dois utilizadores fisicamente próximos, mas de ISPs diferentes sem *peering* direto, pode dar uma “volta” enorme até subir e descer a hierarquia.

Atrasos, perdas e *throughput*

Nesta secção estuda-se **como e porque** os pacotes sofrem atrasos e podem perder-se numa rede de comutação de pacotes, e o que significa “débito” (*throughput*).

Os pacotes são colocados em **filas de espera** nos *routers*: se a taxa de chegada de pacotes a uma ligação de saída excede momentaneamente a capacidade dessa ligação, os pacotes ficam à espera da sua vez. Se não houver espaço livre na fila quando chega um novo pacote, este é **descartado** (perda).

As quatro fontes de atraso

Definição — As quatro componentes do atraso

O atraso total que um pacote sofre entre dois nós consecutivos ($d_{n,n+1}$) é a soma de quatro componentes:

$$d_{n,n+1} = d_{proc} + d_{fila} + d_{trans} + d_{prop}$$

1. d_{proc} — **atraso de processamento**: tempo que o nó gasta a verificar erros no cabeçalho do pacote e a determinar para que ligação de saída o deve encaminhar. Tipicamente alguns microssegundos ou menos.
2. d_{fila} — **atraso na fila de espera**: tempo que o pacote espera na fila antes de poder começar a ser transmitido na ligação de saída — é a soma dos tempos de transmissão de todos os pacotes que estão à frente na fila. Depende do nível de congestionamento do *router* naquele instante (pode variar muito, de quase zero a muito elevado).
3. d_{trans} — **atraso de transmissão**: $= L/R$, onde L é o comprimento do pacote em bits e R a capacidade da ligação em bits/s. É o tempo que demora a “colocar” todos os bits do pacote na ligação. Significativo em ligações de baixa capacidade.
4. d_{prop} — **atraso de propagação**: $= l/v$, onde l é o comprimento físico do meio (distância) e v a velocidade de propagação do sinal nesse meio (tipicamente perto de 2×10^8 m/s, cerca de 2/3 da velocidade da luz no vácuo). Pode ir de alguns microssegundos a centenas de milissegundos (por exemplo, num satélite geossíncrono).

Atenção

Nota adicional (não estava explícito nos slides): atraso de transmissão vs. atraso de propagação — não confundir! São conceitualmente muito diferentes, apesar de ambos dependerem da ligação:

- O **atraso de transmissão** (L/R) é o tempo que o **emissor** demora a empurrar todos os bits do pacote para dentro da ligação — depende do tamanho do pacote e da capacidade da ligação, **não** da distância. Pensa nisto como o tempo que um camião demora a **entrar** todo numa autoestrada através do acesso (depende do comprimento do camião e da “largura” do acesso).
- O **atraso de propagação** (l/v) é o tempo que o **sinal** demora a viajar fisicamente do emissor até ao recetor, depois de já estar todo “na ligação” — depende apenas da distância e da velocidade de propagação no meio, **não** do tamanho do pacote nem da capacidade da ligação. É o tempo que o camião (já todo na autoestrada) demora a percorrer a distância até à saída.

Um pacote muito grande numa ligação muito lenta pode ter atraso de transmissão dominante (minutos); uma ligação via satélite geossíncrono tem atraso de propagação dominante (~270ms), mesmo para um pacote minúsculo. São independentes um do outro.

Atraso com múltiplos pacotes e ligações em série

Quando o percurso passa por várias ligações de capacidades diferentes, e são enviados vários pacotes seguidos (não só um), o comportamento depende muito de **qual ligação é o “estrangulamento”** (a mais lenta do percurso — *bottleneck link*).

Exemplo resolvido

Configuração. Um terminal T1 envia 4 pacotes, cada um com $L = 1000$ bits, para um terminal T2, passando por um único *router* R1: primeiro numa ligação LAN (T1 a R1) e depois numa ligação WAN (R1 a T2). A velocidade de propagação em ambas é $v = 2 \times 10^8$ m/s. Vamos calcular o atraso total até o **último** (4º) pacote chegar completo a T2, em dois cenários: (a) a LAN é a ligação mais lenta; (b) a WAN é a mais lenta.

Fórmulas gerais (com $d_{proc} = d_{fila} = 0$, ou seja, sem congestionamento adicional):

- Se $R_{LAN} < R_{WAN}$ (LAN é o estrangulamento):

$$d_{total} = 4 \cdot d_{trans,LAN} + d_{prop,LAN} + d_{trans,WAN} + d_{prop,WAN}$$

- Se $R_{LAN} > R_{WAN}$ (WAN é o estrangulamento):

$$d_{total} = d_{trans,LAN} + d_{prop,LAN} + 4 \cdot d_{trans,WAN} + d_{prop,WAN}$$

Porque é que o “4×” muda de posição: o fator 4 aplica-se sempre à ligação **mais lenta**. Isto acontece porque essa ligação é o gargalo: os 4 pacotes só conseguem ser colocados nela um a seguir ao outro (a ligação está sempre ocupada com o pacote anterior), pelo que o **último** pacote só começa a ser transmitido depois dos outros 3 já terem sido totalmente transmitidos nessa ligação — daí $4 \times d_{trans}$ **da ligação lenta** determinar quando o último pacote sai dela. Na ligação rápida, como cada pacote é transmitido depressa, não há essa acumulação significativa — conta-se apenas 1 atraso de transmissão (o suficiente para “abrir espaço” ao fluxo de pacotes).

Exemplo resolvido

Cenário (b) com números concretos — WAN é o estrangulamento. Seja $R_{LAN} = 10$ Mbps, $R_{WAN} = 1$ Mbps, $l_{LAN} = 1$ km, $l_{WAN} = 1000$ km, $L = 1000$ bits por pacote. Atrasos elementares:

$$d_{trans,LAN} = \frac{1000}{10 \times 10^6} = 0,1 \text{ ms} \quad d_{prop,LAN} = \frac{1000}{2 \times 10^8} = 0,005 \text{ ms}$$

$$d_{trans,WAN} = \frac{1000}{1 \times 10^6} = 1 \text{ ms} \quad d_{prop,WAN} = \frac{1\,000\,000}{2 \times 10^8} = 5 \text{ ms}$$

Traçando a passagem de cada pacote, um a um (sem saltar nenhum):

Pacote	Sai da LAN (fim transmissão)	Chega a R1 (+ prop. LAN)	Início transmissão WAN (máx. entre chegada e WAN livre)	Fim transmissão WAN	Chega a T2 (+ prop. WAN)
1	0,1 ms	0,105 ms	0,105 ms	1,105 ms	6,105 ms
2	0,2 ms	0,205 ms	máx(0,205; 1,105) = 1,105 ms	2,105 ms	7,105 ms
3	0,3 ms	0,305 ms	máx(0,305; 2,105) = 2,105 ms	3,105 ms	8,105 ms
4	0,4 ms	0,405 ms	máx(0,405; 3,105) = 3,105 ms	4,105 ms	9,105 ms

Repara como, a partir do pacote 2, o “início da transmissão na WAN” já não é logo que o pacote chega da LAN — tem de esperar que a WAN, que é lenta, acabe de transmitir o pacote anterior (é a fila de espera a formar-se no *router* R1, por causa do estrangulamento na WAN). O último pacote (4) chega a T2 aos **9,105 ms**.

Confirmação pela fórmula:

$$d_{total} = 0,1 + 0,005 + 4 \times 1 + 5 = \mathbf{9,105 \text{ ms}}$$

Bate certo com a última linha da tabela.

Exemplo resolvido

Cenário (a), para comparação — LAN é o estrangulamento. Trocando as capacidades: $R_{LAN} = 1$ Mbps, $R_{WAN} = 10$ Mbps (mesmas distâncias e tamanho de pacote de antes).

$$d_{trans,LAN} = 1 \text{ ms} \quad d_{prop,LAN} = 0,005 \text{ ms}$$

$$d_{trans,WAN} = 0,1 \text{ ms} \quad d_{prop,WAN} = 5 \text{ ms}$$

Agora é a **LAN** que acumula os pacotes (a WAN, sendo rápida, escoar-os assim que chegam). Pela fórmula do cenário (a):

$$d_{total} = 4 \times 1 + 0,005 + 0,1 + 5 = \mathbf{9,105 \text{ ms}}$$

(Dá o mesmo valor do cenário anterior por coincidência dos números escolhidos — o que importa reter não é o valor final, mas **qual ligação recebe o fator 4**: é sempre a mais lenta, porque é aí que os pacotes se acumulam.)

Intensidade de tráfego e explosão do atraso

Definição

Seja R a capacidade da ligação (b/s), L o comprimento médio dos pacotes (bits) e a a taxa média de chegada de pacotes (chegadas independentes umas das outras). Define-se a **intensidade de tráfego** (ou tráfego oferecido) como La/R — a fração da capacidade da ligação que, em média, está a ser pedida.

- $La/R \approx 0$: a fila está quase sempre vazia, tempo médio de espera mínimo.
- $La/R \rightarrow 1$: o tempo médio de espera na fila **aumenta muito rapidamente** (a curva não é linear — cresce de forma explosiva perto de 1).
- $La/R > 1$: chega, em média, mais “trabalho” do que aquele que a ligação consegue escoar — o tempo médio de espera tende para **infinito** (a fila cresce sem limite, na teoria; na prática a fila tem capacidade finita, e os pacotes começam a perder-se — ver secção seguinte).

Atenção

Nota adicional (não estava explícito nos slides): este comportamento não-linear é uma propriedade geral de sistemas de filas de espera com chegadas aleatórias (é estudado em teoria das filas, por exemplo em modelos M/M/1) — a ideia chave para reter é que **não há uma relação proporcional simples** entre “quão perto de 100% de utilização” está uma ligação e “quanto atraso” isso causa. Utilizar uma ligação a 95% de capacidade pode já implicar atrasos de fila muito maiores do que utilizá-la a 80%, e não apenas ligeiramente maiores. É por isso que, na prática, se tenta manter as ligações de rede com alguma margem de folga, em vez de as operar sempre no limite da capacidade.

Atrasos reais: traceroute

O programa **traceroute** mede o atraso real desde a origem até cada *router* ao longo do percurso para um destino. Para cada *router* i no caminho, envia 3 pacotes que chegarão a esse *router*; o *router* i devolve uma mensagem de resposta à origem; o emissor mede o tempo entre o envio e a chegada da resposta (para cada um dos 3 pacotes, dando 3 medições por salto).

Exemplo resolvido

Como ler uma saída de traceroute. No exemplo dos slides (**traceroute** do DCC/FCUP até www.cmu.edu), cada linha tem: número do salto, as 3 medições de atraso (em ms), e o nome/IP do *router* nesse salto. Coisas a reparar:

- Os primeiros saltos (dentro da rede da universidade e da rede académica portuguesa/europeia) têm atrasos muito baixos (1-6 ms) — são saltos fisicamente próximos.
- Entre os saltos 10 e 11, o atraso salta de ~32ms para ~102ms — um aumento súbito de ~70ms. Isto é o sinal característico de uma **ligação trans-oceânica** (fibra submarina Europa-EUA): a distância é muito maior, logo o atraso de **propagação** (l/v) aumenta muito nesse único salto (não é o processamento nem a fila que sobem — é a distância física).
- A linha com * * * **Request timed out** significa que não houve resposta a nenhum dos 3 pacotes enviados àquele *router* naquele salto — pode ser porque o *router* está configurado para não responder a este tipo de pedido, ou porque o pacote (ou a resposta) se perdeu. Não significa necessariamente que a rede esteja quebrada ali — o **traceroute** normalmente continua a conseguir sondar os saltos seguintes.

Perdas de pacotes

Como a fila de espera de um *router* tem **capacidade finita**, um pacote que chegue e encontre a fila cheia é **descartado** — é uma perda de pacote. Um pacote perdido pode:

- ser retransmitido pelo nó anterior (se esse nó implementar essa garantia — nem todos os protocolos de ligação o fazem);

- ser retransmitido pelo terminal de origem (é o que o TCP faz, ao nível de transporte, detetando a perda e reenviando);
- ou simplesmente não ser retransmitido (aceitável para tráfego que usa UDP, como vídeo em tempo real, em que chegar tarde não compensa).

Throughput (débito)

Definição — Débito (throughput)

O **débito** é a taxa a que os bits são efetivamente transferidos entre o emissor e o recetor. Distingue-se o débito **instantâneo** (num instante específico) do débito **médio** (ao longo de um período de tempo mais alargado).

Se pensarmos na transferência de dados como um fluido a passar por um “tubo”: se o servidor envia dados a uma taxa R_s e o cliente os recebe a uma taxa R_c , e essas taxas forem diferentes ao longo do percurso, o débito efetivamente observado é limitado pelo **elo mais lento** do percurso — a **ligação de estrangulamento** (*bottleneck link*): a ligação, ao longo de todo o caminho, que limita o débito final do fluxo (independentemente de R_s ou R_c serem maiores, se houver algures no meio uma ligação mais lenta, é ela que dita o débito).

Exemplo resolvido

Débito com várias conexões a partilhar um *backbone*. Suponha-se que existem 10 conexões, cada uma entre um servidor com débito de acesso $R_s = 2$ Mbps e um cliente com débito de acesso $R_c = 2$ Mbps, mas todas as 10 conexões partilham de forma justa uma ligação de *backbone* comum com capacidade $R = 10$ Mbps.

O débito por conexão é dado por $\min(R_c, R_s, R/10)$ — o mínimo entre a capacidade de acesso do cliente, a do servidor, e a fatia do *backbone* que cabe a cada uma das 10 conexões (assumindo partilha igual entre todas):

$$\min(2, 2, 10/10) = \min(2, 2, 1) = 1 \text{ Mbps}$$

Ou seja, apesar de cliente e servidor terem, cada um, capacidade para 2 Mbps, o débito real fica limitado a 1 Mbps por conexão — o *backbone* partilhado é que é, aqui, a ligação de estrangulamento. Isto mostra por que motivo, na prática, R_c ou R_s (as ligações de acesso, mais próximas do utilizador) nem sempre são a ligação de estrangulamento — o gargalo pode estar bem mais “no meio” da rede, num ponto partilhado por muitos fluxos ao mesmo tempo.

Ligação com a prática: os Exercícios 2 a 6 da Semana 1 aplicam diretamente as fórmulas de atraso desta secção (transmissão, propagação, *store-and-forward*, débito) — em particular, o Exercício 4 é estruturalmente idêntico ao exemplo do cenário LAN/WAN acima (uma ligação LAN seguida de uma WAN, com pedido explícito de diagrama temporal — a tabela do exemplo acima é exatamente esse tipo de diagrama, só que para um conjunto diferente de valores). O Exercício 1, sobre encaminhamento em redes muito grandes, liga-se à ideia de hierarquia de ISPs e encaminhamento hierárquico introduzida acima (o encaminhamento hierárquico propriamente dito só será aprofundado na camada de rede, mais para a frente no semestre). O Exercício 5 pede o “produto largura de banda-atraso” — ver a nota adicional abaixo, que introduz este conceito (não veio explícito nos slides desta aula, mas é necessário para o exercício).

Atenção

Nota adicional (não estava explícito nos slides): produto largura de banda-atraso. Define-se como $R \cdot d_{prop}$ — o produto entre a capacidade da ligação e o atraso de propagação. Fisicamente, representa o **número máximo de bits que podem estar “em trânsito” na ligação num dado instante**, antes de o primeiro bit enviado chegar ao recetor — é como perguntar “quantos bits cabem dentro do tubo, se o tubo tivesse o comprimento da ligação?”.

Exemplo: uma ligação com $R = 100$ Mbps e um atraso de propagação de $d_{prop} = 15$ ms (por exemplo, uma ligação de fibra de ~ 3000 km, com $v = 2 \times 10^8$ m/s: $d_{prop} = 3000000/(2 \times 10^8) = 0,015$ s). O produto largura de banda-atraso é:

$$R \cdot d_{prop} = 100 \times 10^6 \text{ b/s} \times 0,015 \text{ s} = 1\,500\,000 \text{ bits} = 1,5 \text{ Mbit}$$

Isto significa que, no instante em que o primeiro bit enviado está prestes a chegar ao recetor, o emissor já pode ter colocado até 1,5 Mbit “dentro do tubo”, ainda a caminho, sem que nenhuma confirmação de receção tenha ainda voltado. Este conceito será central mais tarde, quando se estudar o dimensionamento da janela de transmissão do TCP (para o emissor conseguir “encher o tubo” e maximizar o débito, sem desperdiçar capacidade à espera de confirmações).

Camadas protocolares e modelos de serviço

Porquê estruturar em camadas?

As redes são sistemas complexos: muitas “peças” diferentes (terminais, *routers*, ligações de tipos diferentes, aplicações, protocolos, hardware, software). A pergunta natural é: como tornar essa complexidade tratável?

Exemplo resolvido

Analogia: produção e venda de roupa. Pensa numa cadeia de produção de roupa: produção das fibras (matéria-prima) $\rightarrow$ fabricação das peças $\rightarrow$ *design*, *branding* e controlo de qualidade $\rightarrow$ venda a retalho $\rightarrow$ cliente. Cada camada depende apenas do serviço fornecido pela camada imediatamente abaixo (a loja de retalho não precisa de saber como as fibras são produzidas, só precisa que a fábrica lhe entregue peças de roupa acabadas). É uma série de passos desde a matéria-prima até ao consumidor, organizados numa **estrutura por camadas**.

Vantagens de organizar um sistema complexo em camadas:

- Uma estrutura explícita permite identificar e relacionar as diferentes peças do sistema — serve como **modelo de referência** para discussão (todos falam da “camada de transporte” e sabem do que se trata, sem ambiguidade).
- A modularização facilita a manutenção e atualização do sistema: alterações na **implementação** de uma camada são transparentes para o resto do sistema, desde que o **serviço** que ela presta às camadas vizinhas se mantenha igual (tal como uma mudança na organização interna de uma fábrica não afeta o resto da cadeia de produção de roupa, desde que continue a entregar as mesmas peças).

A pilha protocolar da Internet

Definição — As 5 camadas da pilha da Internet

De cima para baixo:

1. **Aplicação:** suporta as aplicações em rede — protocolos como FTP, SMTP, HTTP.
2. **Transporte:** transferência de dados entre *processos* (não só entre máquinas) — protocolos TCP e UDP (ver secção “Extremidade da rede” acima).
3. **Rede:** encaminhamento dos **datagramas** desde a origem até ao destino — protocolo IP e protocolos de encaminhamento.
4. **Ligação de dados:** transferência de dados **direta** entre elementos de rede adjacentes (ex: entre um terminal e o *switch* a que está ligado, ou entre dois *routers* vizinhos) — Ethernet, WiFi, PPP.
5. **Física:** os sinais no meio físico que representam os *bits*.

O modelo de referência ISO/OSI

O modelo OSI tem **7 camadas** — as mesmas 5 da Internet, mais duas entre a aplicação e o transporte:

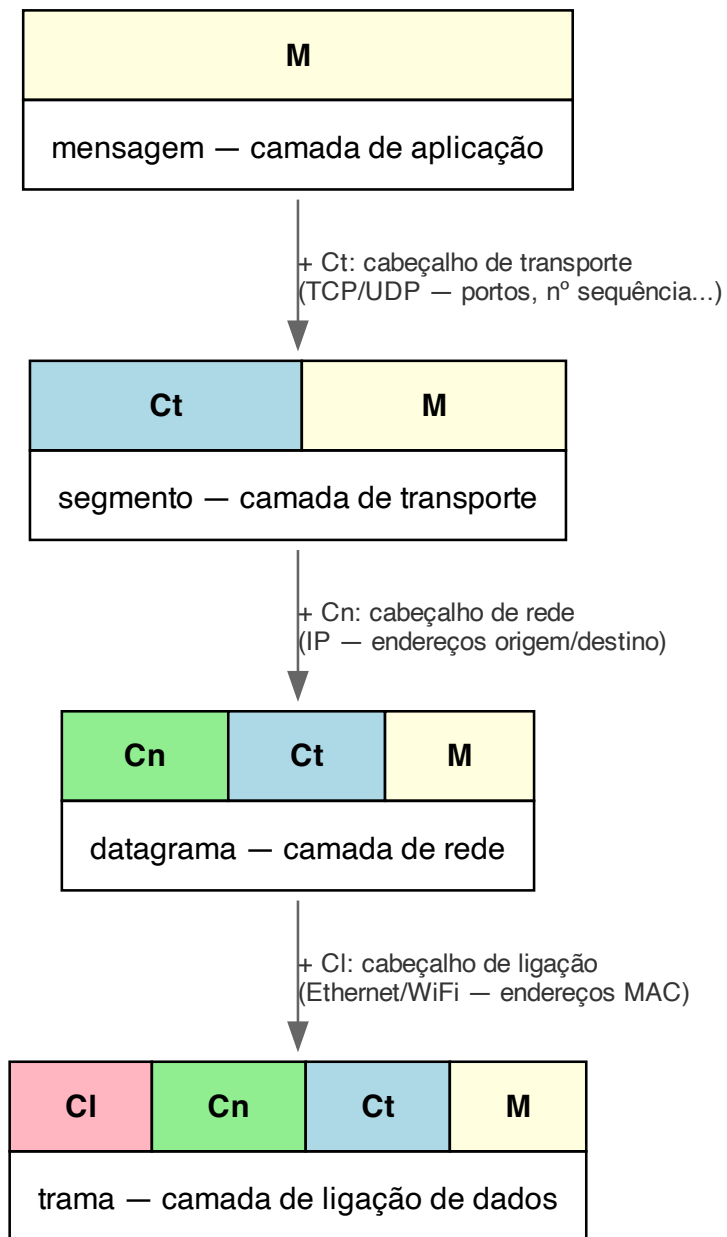
- **Apresentação:** permite às aplicações interpretar o significado dos dados — por exemplo, compressão, cifragem, convenções específicas da máquina (como representar números, texto, etc.).
- **Sessão:** autenticação, sincronização, *checkpointing* e recuperação de sessões.

Atenção

Nota adicional (não estava explícito no slide, mas foi dito explicitamente que é importante): a pilha da Internet **não tem** estas duas camadas (apresentação e sessão) como camadas próprias e separadas. Se uma aplicação precisar destes serviços (por exemplo, cifrar os dados antes de enviar, como faz o HTTPS), esses serviços têm de ser **implementados dentro da própria camada de aplicação** — não existe uma camada dedicada na pilha da Internet que os forneça automaticamente a todas as aplicações. É uma diferença de design deliberada entre os dois modelos, e é um ponto fácil de confundir num exame (perguntar “que camada do modelo OSI corresponde à camada de aplicação da Internet?” não tem uma resposta de um-para-um simples, porque a aplicação da Internet cobre também o que no OSI seria apresentação e sessão).

Encapsulamento

Quando uma mensagem desce a pilha protocolar no terminal de origem, cada camada acrescenta o seu próprio **cabeçalho** de controlo aos dados recebidos da camada acima — é o processo de **encapsulamento**. No terminal de destino, o processo é inverso: cada camada remove o seu cabeçalho antes de entregar os dados à camada de cima.



Exemplo resolvido

Seguindo uma mensagem pela rede, passo a passo. A camada de aplicação do terminal de origem entrega uma **mensagem** (M) à camada de transporte. A camada de transporte acrescenta o seu cabeçalho (C_t — por exemplo, os números de porto de origem/destino do TCP), formando um **segmento** ($C_t M$), e entrega-o à camada de rede. A camada de rede acrescenta o seu cabeçalho (C_n — os endereços IP de origem e destino), formando um **datagrama** ($C_n C_t M$), e entrega-o à camada de ligação de dados. A camada de ligação acrescenta o seu cabeçalho (C_l — os endereços MAC de origem e destino), formando uma **trama** ($C_l C_n C_t M$), pronta para ser enviada como sinais na camada física. Ao longo do percurso, os dispositivos intermédios só “abrem” os cabeçalhos de que precisam para fazer o seu trabalho: um **switch** (dispositivo da camada de ligação de dados) só olha para o cabeçalho C_l , para decidir por que porta física reenviar a trama — não sabe nem precisa de saber o que está dentro do datagrama IP. Um **router** (dispositivo da camada de rede) tem de “abrir” a trama (removendo C_l) para chegar ao datagrama e ler o cabeçalho C_n (o endereço IP de destino), decidir para que ligação de saída o deve reenviar, e depois **voltar a encapsular** o datagrama numa nova trama (com um novo C_l , apropriado à ligação de saída seguinte) — é por isso que os endereços MAC mudam a cada salto ao longo do percurso, enquanto os endereços IP (dentro de C_n) se mantêm os mesmos do início ao fim. Só no terminal de destino é que todos os cabeçalhos são sucessivamente removidos (ligação $\rightarrow$ rede $\rightarrow$ transporte) até restar apenas a mensagem M original, entregue à aplicação de destino.

Frequente em exame

Um erro comum é pensar que o *router* olha para o cabeçalho de **transporte** (C_t) para decidir o encaminhamento — não olha: o encaminhamento na camada de rede é feito exclusivamente com base no cabeçalho de **rede** (C_n , endereço IP de destino). Da mesma forma, um *switch* nunca olha para o endereço IP — só usa endereços MAC (camada de ligação). Esta separação de responsabilidades por camada é um ponto frequentemente testado.